## Processor Trends

**Example**

**A story of nonlinear growth** (Ed Catmull, 2020 ACM Turing Award)
In 1979 Catmull wanted to make a digital movie : it would have needed the power of 1000 CRAY-1 supercomputers (10 million US$ each), with a budget of 1M$. Based on the exponential curve of computing speed, he estimated it would take another 14-15 years : "Toy Story" came out about 15 years later, very close to the estimate.

**Key idea**

50 years of microprocessor trends ("Moore's Law") : the number of transistors keeps growing, but frequency and single-thread performance have flattened.

- We hit a **power wall at ~100 W/cm$^2$** : frequency is fixed because raising it increases power usage & temperature.
- Our processors aren't getting much faster anymore.
- The only thing we can do is to have **more processors** (multi-core CPUs, GPUs).

**Summary**

**Why this history lesson ?**

- The only way to get good performance nowadays is to anticipate how an algorithm will run on a particular piece of hardware and then *design for it*.
- Sometimes : use a good implementation of a common building block (e.g. a matrix factorization by Intel/NVIDIA). When your needs are more special, you will likely have to make it yourself.
- Order-of-magnitude speedups are easy to get.

**Warning**

You don't have to learn C++ or assembly code for the CS328 final exam, but you should know that these things exist and how they *roughly* work.

## Model Problem : psum

**Definition**

**psum(x)** : sum the positive elements of the input vector `x`. A simple operation, but one that isn't "natively" supported by NumPy.

**Tip of the iceberg** : below the Python code (browser, Jupyter, NumPy, bytecode interpreter, OS) lie instruction decoding, Tomasulo's algorithm, speculative execution, caches and prefetch, DRAM, physical bits. On a GPU : CUDA/Vulkan, kernels, hardware scheduler, shared memory, latency hiding.

**Python bytecode** : `dis.dis(psum)` shows the stack-machine instructions (`LOAD_FAST`, `BINARY_SUBSCR`, `COMPARE_OP`, ...).

**Key idea**

**Dissecting the interpreter** : a big loop fetches the next *opcode* and jumps to one of ~130 code blocks. `COMPARE_OP` pops two operands, calls `PyObject_RichCompare` (stack overflow check, type dispatch, reference counting, error management), down to `float_richcompare` ("Comparison is pretty much a nightmare").

- That's *a lot* of code for something so simple : the overhead dwarfs the actual operation.
- Inherent compromise of an interpreted language. Overheads are usually negligible when the operation does more work.

**Example**

**Vectorization in Python**
Bad : a Python loop `if B[i] < 0: X[i] = A[i] + C[i] else: X[i] = A[i] + D[i]`
Good : `X = A + np.where(B < 0, C, D)`

## First Version in C++

**Example**

```
float psum_0(float *x, size_t size) {
  float r { 0.f };
  for (size_t i = 0; i < size; ++i) {
    if (x[i] >= 0) {
      r += x[i];
      if (std::isinf(r))
        abort();
    }
  }
  return r;
}
```

Everything has types : `float` (single precision), `size_t` (64-bit integer), `float*` (pointer to a float array). Glued to Python with **nanobind**.

**Definition**

**DISASSEMBLER**
Reveals the processor instructions ("assembly code") in a compiled executable (IDA Home, Ghidra). Used here to understand the performance differences between `psum` versions.

The disassembly of `psum_0` shows a loop of `LDR` (load), `FCMP` + `B.LT` (compare and branch), `FADD`, the infinity check (`FABS`, `FCMP`, `B.EQ`), then the loop counter.

## Pipelining, Latency and Throughput

**Example**

**Laundry day** : 3 batches (Dark, Light, Color), 3 steps (Wash, Dry, Fold). Naive schedule : 9 cycles. Better (pipelined) schedule : **4 cycles**.

**Definition**

**LATENCY** : number of cycles before the result of an operation is available.
**THROUGHPUT** : number of operations that can be started per cycle.

**Key idea**

**Pipelining within processing units** : a unit split into 5 stages has latency 5 but throughput 1 (one new input per cycle); with 5 independent inputs it keeps all stages busy.

**Key formulas**

**Intel "Ice Lake"** (1 cycle on a 3 GHz CPU = 0.33 ns; light moves 10 cm)

| Instruction | Latency | Throughput |
|---|---|---|
| `int64` integers | | |
| add, sub | 1 | 4 |
| mul | 3 | 1 |
| div | 15 | 1/14 |
| `float32` vectors | | |
| add, sub, mul | 4 | 2 |
| div | 17 | 1/10 |

Throughput 4 : can start 4 additions every cycle. 1/14 : busy for 14 cycles before new input can be accepted. **Loading memory, division and square root are really expensive.**

## Superscalar Execution

**Definition**

**TOMASULO'S ALGORITHM** ("superscalar execution")
Aggressively parallelizes the execution of a program : instead of running it step by step, try to do everything at once while respecting data dependencies. Processing units must wait for their inputs to be ready ("reservation stations").

- LOADs are **extremely** slow : helpful to do multiple in parallel.
- Newer architectures (e.g. Apple M1) look at the eight next instructions per clock cycle; in a loop, they can run ahead several iterations.
- The CPU is like a high-speed train : to keep the pipeline fed, it must look far ahead (decode, schedule and execute different instructions at the same time).

**Definition**

**BRANCH PREDICTION**
At each branch, the processor must decide what to do a long time in advance (waiting would be too slow) : ask the oracle (jump or not ?), go that way and begin executing; N cycles later we know if it was right. Correct : all good. Wrong : pull the emergency brake and undo all steps.

**Warning**

In the naive `psum_0`, mispredictions happen all the time : the input values are random and positive with 50% probability.

**Example**

**Improved version** (no branch) : `for (...) r += fmaxf(x[i], 0.f);` – compiled to `FMAXNM` + `FADD`.

## SIMD and Partial Sums

**Definition**

**SIMD** (single instruction, multiple data)
CPUs can process vectors : in the same amount of time, add $x[0]+y[0]$ or 16 elements at once. Accessible through **intrinsics** (Intel/AMD : SSE, AVX, AVX512 ; ARM : NEON, SVE).

**Warning**

The idea was that compilers would magically insert vector instructions ("auto-vectorization"). It never really worked, and still does not work well enough.

**Key idea**

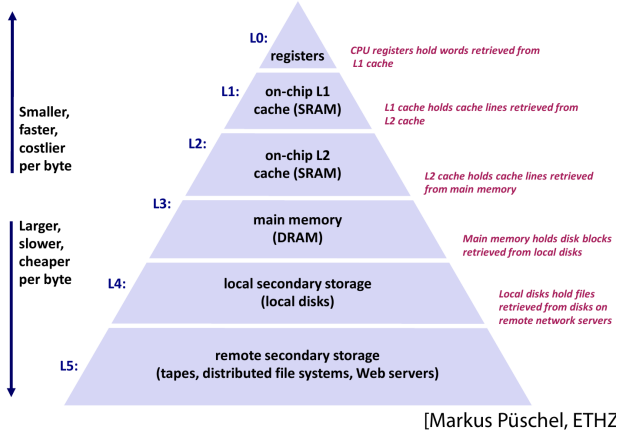
**What's missing ?** Each SUM depends on the previous SUM : the program is still not parallel enough to keep the pipeline fed. Break these connections : compute **4 (or 8 or 16) partial sums** and add them up at the end (`psum_3` : 4 vector accumulators).

## Memory & Caches

**Key formulas**

**Load instruction** : latency ∼5-300 cycles, throughput 2. Depends on where we go in the memory hierarchy : registers, L1, L2 (SRAM), main memory (DRAM), local disks, remote storage – smaller, faster, costlier per byte at the top.

**Typical Memory Hierarchy**




[Markus Püschel, ETHZ

**Definition**

**Temporal locality** : the same memory is accessed again soon.
**Spatial locality** : nearby memory is accessed soon.
**Prefetchers** exploit spatial & temporal regularity : after a few misses on a regular access pattern, loads become hits.

**Example**

**Cache hit** : get something predictable from the corner store. **Cache miss** : weird unpredictable access, wait a long time (but can order multiple packages in parallel).

**Key idea**

**CPUs vs GPUs** : multiple instructions per cycle, huge caches, branch prediction, speculative execution… GPUs don't have it! A CPU is a cheetah; a GPU is an army of snails : each core much simpler, smaller and slower, but lots of them.

**Summary**

**General advice** – when optimizing, be wary of

— sacrificing correctness for speed (ensure correctness first, e.g. tests);

— reinventing the wheel (use the right tools, libraries);

— valuing computer time over your own time (maybe optimization isn't needed).

**Always measure** : most time is spent in a few bottlenecks; locate them and focus on them.